



TÓPICOS EM INFORMÁTICA

Scene 2D, user interface e skins



SCENE 2D

- Scene 2D, na libgdx, é um **grafo 2D de cena** utilizado para construir aplicações, interfaces gráficas com o usuário usando uma **hierarquia de atores**. A scene 2D fornece as seguintes features:
 - Operações de **rotação e escala** que, se aplicadas a um ator é aplicada também a todos os atores filhos.
 - O desenho das interfaces é feito por meio do **SpriteBatch**.
 - **Deteção de hit e clique** (inclusive de atores rotacionados ou aumentados com scale/escala).
 - Envio ou roteamento de **dados de entrada a atores apropriados**.
 - **Sistema de ações** para manipulação de atores ao longo do tempo. Autores podem ser combinados para obter efeitos complexos.



ACTORS (ATORES)

- Esses grafos de cena **não seguem o padrão MVC do android.**
- Os atores guardam informações **tais como tamanho e posição** (não é MVC).
 - Atores também são a View, uma vez que eles sabem se desenhar.
- MVC é a arquitetura **Model-View-Controller**, onde a definição do desenho da parte gráfica está **destacada** do modelo e do controle (que são código da aplicação).
 - No android, por exemplo, isso é feito por meio de arquivos **XML**.

CLASSES DA SCENE 2D

- Basicamente, Scene 2D (que pode ser visto como um pacote), tem um total de três classes em seu núcleo, sendo:
 - A classe **Actor**.
 - É um nó no grafo que tem **posição**, **tamanho retangular**, **origem** (para **operações de rotação**, etc), **escala**, **rotação** e **cor**.
 - A classe **Group**.
 - A classe grupo é um ator que pode **ter atores filhos**.
 - A classe **Stage**.
 - Uma tradução literal aqui seria **palco** ou **cena**. A classe **Stage** tem uma **câmera**, um **spritebatch**, e **controla o desenho dos atores e a distribuição das inputs / entradas** no seu sistema de interface.



STAGE (PALCO)

- **Stage** é um **InputProcessor**.
 - Quando **Stage** recebe eventos de entrada ele **direciona para os atores apropriados**.
 - Caso o evento tenha sido tratado por algum Actor no Stage, o **InputProcessor do Stage retornará true** (que indica que o input foi tratado e para a cadeia de tratamento do input – evitando **cliques em dois botões** que estejam na mesma posição, por exemplo).
- Stage tem um **método act**.
 - Esse método recebe delta e **chama todos os métodos act dos atores**, permitindo que os elementos da sua composição gráfica recebam o tempo do jogo e possam animar, movimentar, ou qualquer coisa do tipo.



DRAWING (DESENHANDO)

- Quando o método **draw** do Stage é chamado, isso chama o **draw** de todos os **Actors** nesse **Stage**.
- Existe um método chamado **setVisible** dos atores.
 - Caso receba false, o método **draw** do ator específico não será chamado.
- Se um ator **precisar fazer algum desenho que não seja com SpriteBatch**, como, por exemplo, com um **ShapeRenderer**, é necessário **encerrar o Batch e começá-lo novamente depois**.
 - Código exemplo no próximo slide.



DRAWING (DESENHANDO)

```
private ShapeRenderer renderer = new ShapeRenderer();
```

```
public void draw (Batch batch, float parentAlpha) {  
    batch.end();  
  
    renderer.setProjectionMatrix(batch.getProjectionMatrix());  
    renderer.setTransformMatrix(batch.getTransformMatrix());  
    renderer.translate(getX(), getY(), 0);  
  
    renderer.begin(ShapeType.Filled);  
    renderer.setColor(Color.BLUE);  
    renderer.rect(0, 0, getWidth(), getHeight());  
    renderer.end();  
  
    batch.begin();  
}
```

TRANSFORMAÇÃO DE GRUPO

- Quando o grupo é **rotacionado ou escalado**, todos os filhos são desenhados normalmente mas o **SpriteBatch se encarrega de desenhá-los rotacionados e escalados**.
- Para acelerar a computação, é possível chamar o método **setTranform(false)** da classe **Group** para pular essa etapa caso os atores não estejam rotacionados ou escalados.

HIT (COLISÃO)

- Existe um método chamado **hit** na classe Actor. O método **hit** recebe um ponto (coordenadas x e y) e retorna o **primeiro ator que corresponde àquele ponto**, ou null caso não exista uma colisão com um ator.

```
public Actor hit (float x, float y, boolean touchable) {  
    if (touchable && getTouchable() != Touchable.enabled) return null;  
    return x >= 0 && x < width && y >= 0 && y < height ? this : null;  
}
```

- O método hit pode ser sobrescrito (**herdado e overridden**) caso o ator seja redondo, por exemplo.
- Você também pode **desabilitar o “touch” do botão**, para o mesmo não ser retornado, caso necessário.



EVENTOS

- **EventListeners** são adicionados aos atores e são chamados quando algum evento ocorre. Por exemplo, o **InputListener** pode ser usado para tratar eventos de entrada:

//limites do "botão", por exemplo

```
actor.setBounds(0, 0, texture.getWidth(), texture.getHeight());
```

```
actor.addListener(new InputListener() {  
    public boolean touchDown (InputEvent event, float x, float y, int pointer, int button) {  
        System.out.println("down");  
        return true;  
    }  
  
    public void touchUp (InputEvent event, float x, float y, int pointer, int button) {  
        System.out.println("up");  
    }  
});
```



EVENTOS

- Para tratar movimento de mouse, faça o override (sobrescreva) o método `mouseMoved`.
- Para tratar se um mouse entrou ou saiu de um ator sobrescreva o método `enter` e / ou `exit`.
- Para tratar eventos do teclado sobrescreva `keyDown`, `keyUp` ou `keyTyped`.
 - Esses métodos vão ser chamados apenas para o ator que tenha foco do keyboard. Você pode trocar o foco chamando o método `setKeyboardFocus`.
- Se `setTouchable(false)` or `setVisible(false)` forem chamados, o ator não receberá as chamadas do evento de entrada.



OUTROS LISTENERS

- Também é possível utilizar outros Listeners da seguinte forma:

```
actor.addListener(new ActorGestureListener() {  
    public boolean longPress (Actor actor, float x, float y) {  
        System.out.println("long press " + x + ", " + y);  
        return true;  
    }  
  
    public void fling (InputEvent event, float velocityX, float velocityY, int button) {  
        System.out.println("fling " + velocityX + ", " + velocityY);  
    } //lembrando que o fling é o "sweep" ou gesto de arrastar  
  
    public void zoom (InputEvent event, float initialDistance, float distance) {  
        System.out.println("zoom " + initialDistance + ", " + distance);  
    }  
  
});
```

ACTIONS (AÇÕES)

- Cada ator tem uma lista de ações. As ações são atualizadas na chamada do método `act`. Ações servem para `executar` alguma coisa no clique de um botão, por exemplo.
- Quando a ação termina, ela é **automaticamente** removida do ator.

```
MoveToAction action = new MoveToAction();  
action.setPosition(x, y);  
action.setDuration(duration);  
actor.addAction(action);
```



POOLING DE AÇÕES

- Também é possível fazer o Pooling de ações:

```
Pool<MoveToAction> pool = new Pool<MoveToAction>() {  
    protected MoveToAction newObject () {  
        return new MoveToAction();  
    }  
};
```

```
MoveToAction action = pool.obtain();  
action.setPool(pool);  
action.setPosition(x, y);  
action.setDuration(duration);  
actor.addAction(action);
```

AÇÕES COMPLEXAS

- É possível inserir ações mais complexas **rodando ações em paralelo (ao mesmo tempo) ou em sequência**. A classe **ParallelAction** tem uma lista de ações predefinidas e as roda ao mesmo tempo:

```
actor.addAction(sequence(moveTo(200, 100, 2), color(Color.RED, 6),  
delay(0.5f), rotateTo(180, 5))));
```

AÇÃO DE COMPLETUDE

- Para rodar um código quando uma ação estiver concluída é possível usar uma sequência com um RunnableAction:

```
actor.addAction(sequence(fadeIn(2), run(new Runnable() {  
    public void run () {  
        System.out.println("Action complete!");  
    }  
})));
```




EXERCÍCIO

- Para fins de aprendizado, com as ideias que vimos até agora, **monte as suas classes**:
 - Stage (ou Container)
 - Actor (ou Content)
- Faça o **Container** conter um ou vários **Contents**, com métodos para adicionar ou remover **Contents**.
- Faça o **Container** ter um método draw para desenhar tudo e uma forma de adicionar os input processors dos **Contents** para funcionar.



EXERCÍCIO

- Ainda no mesmo exercício, adicione uma forma (sua forma) de **rodar ações**. Ou seja, associar algum pedaço de código para funcionar no clique de um botão.
- Faça um GUI de exemplo usando essa sua codificação, contendo, **pelo menos**:
 - Dois botões, com ações
 - Uma imagem



INTERFACE DE USUÁRIO (USER INTERFACE)

- Até agora vimos o funcionamento e a lógica da **Scene 2D**, que é a “arquitetura” que rege as interfaces de usuário (**UI** ou **GUI**) da libgdx.
- De forma semelhante ao Android, existem as classes **Layout** na libgdx para **posicionar os atores de forma inteligente** (sem precisar muitas vezes ficar reposicionando manualmente baseado na resolução do cliente).
 - Não vamos entrar muito nesse detalhe mas vocês podem olhar depois **caso tenham interesse**.



TABLE

- A maioria dos layouts usam uma **Table** (tabela) que é do tamanho do **Stage**.
 - Todos os outros widgets gráficos (botões, caixas de texto, etc) e tabelas são colocados “dentro” ou aninhados a essa tabela principal.

```
private Stage stage;  
private Table table;  
  
public void create () {  
    stage = new Stage();  
    Gdx.input.setInputProcessor(stage);  
  
    table = new Table();  
    table.setFillParent(true); // Faz a tabela se alinhar ao  
    tamanho de seu pai, no caso, o Stage  
    stage.addActor(table);  
  
    table.setDebug(true); // Opcional, habilita linhas de debug  
    para tabelas.  
  
    // Adicione os widgets ou elementos gráficos aqui.  
}
```

```
public void resize (int width, int height) {  
    stage.getViewport().update(width, height, true);  
}  
  
public void render () {  
    Gdx.gl.glClear(GL20.GL_COLOR_BUFFER_BIT);  
    stage.act(Gdx.graphics.getDeltaTime());  
    stage.draw();  
}  
  
public void dispose() {  
    stage.dispose();  
}
```



CLIPPING

- Existe um método chamado `setClip(true)` nas `Tables`.
 - Fazer a chamada do método `setando-o para true` habilita o clipping e `culling dos atores`.
- Nesse caso, os atores `dentro da tabela` serão “cortados” ou deixados de desenhar (culling) caso estejam totalmente fora da tela.
 - `Técnicas de culling` são muito utilizadas mesmo no 3D para apagar polígonos que não estão visíveis na câmera.
- Atores podem ser adicionados à tabela utilizando os métodos `add` e `addActor`.



SKIN

- Todos os widgets da UI são feitos de **alguns recursos configuráveis** tais como: imagens, fontes, cores, etc.
 - Todos os recursos necessários para renderizar um widget é chamado de estilo (ou **style**).
- Cada widget define a sua classe de estilo.
 - Existe um **construtor inicial** com definição de estilo ou é possível chamar o método **setStyle** para trocá-lo.



SKIN E STYLE

- Estilos podem ser configurados a partir de JSON ou pelo código:

```
TextureRegion upRegion = ...  
TextureRegion downRegion = ...  
BitmapFont buttonFont = ...
```

```
TextButtonStyle style = new TextButtonStyle();  
style.up = new TextureRegionDrawable(upRegion);  
style.down = new TextureRegionDrawable(downRegion);  
style.font = buttonFont;
```

```
TextButton button1 = new TextButton("Button 1", style);  
table.add(button1);
```

```
TextButton button2 = new TextButton("Button 2", style);  
table.add(button2);
```

EVENTOS DE MUDANÇA

- A maioria dos widgets dispara um **ChangeEvent** quando algo muda. Esse evento é um **evento genérico** (que funciona para diferentes widgets).
 - E.g.: Para um botão o evento é o **pressionar do botão**, para um slider a **mudança é a posição do slider**, etc.
- **ChangeListener** deve ser utilizado para detectar esses eventos:

```
actor.addListener(new ChangeListener() {  
    public void changed (ChangeEvent event, Actor actor) {  
        System.out.println("Changed!");  
    }  
});
```




LAYOUT WIDGETS

- **Table**: a classe Table posiciona os seus widgets filhos de forma similar à tabelas HTML.
 - As tabelas permitem a disposição dos widgets sem utilizar as posições absoluta para os itens (x e y). Portanto, a mesma se adapta a diferentes tamanhos de widgets e resoluções de tela.
- **Container**: é equivalente a uma Table porém com um único filho.
 - É mais leve (performance).
 - **VerticalGroup** e **HorizontalGroup** também são opções parecidas com Table e mais leves.
- **Stack**: é um widget de grupo. Serve para adicionar um widget em cima do outro.
 - O primeiro widget é desenhado embaixo e o último a ser adicionado é desenhado em cima.



LAYOUT WIDGETS

- **ScrollPane**: coloca o filho como se fosse dentro de um iframe (do HTML). Ou, em outras palavras, escrolla o filho com scroll bars.
- **SplitPane**: contém dois widgets e é dividido em dois (ou horizontalmente ou verticalmente).
 - O usuário pode redimensionar os widgets com um splitter (uma barra separadora).
- **Tree**: monta uma hierarquia de nós.
 - Cada nó pode ter filhos e pode ser expandido/contraído. Cada nó tem um ator permitindo flexibilidade na questão de visualização do ator. A Tree tem ícones de + e – perto perto de cada ator dos nós.



WIDGETS

- **Label**: desenha um texto usando **BitmapFont** e uma cor.
 - É possível ativar o **word wrap** (para limitar o tamanho horizontal do texto). Nesse caso, a largura do label deve ser **setada pelo seu pai**.
- **Image**: simplesmente **desenha um drawable**.
 - Drawable pode ser uma textura, texture region, ninepatch, sprite, etc.
- **Button**: desenha um botão.
 - Estende Table para que outros **widgets possam ser inseridos dentro do botão** (um drawable, por exemplo).
 - Possui um background **up** e um background **down**.
 - Tem um estado de “**pressionado**”. Nesse caso específico usará o background **checked** ao invés de **up** e **down** caso definido.



WIDGETS

- **TextButton**: estende a classe **Button** e contém um **Label**.
- **ImageButton**: estende **Button** e contém um **Image** widget.
- **CheckBox**: estende **TextButton** e adiciona um **Image** widget na esquerda do **Label**.
- **ButtonGroup**: não é um ator e não tem visuais.
 - Botões podem ser adicionados a ele e essa classe de grupo **força um número máximo e mínimo de botões pressionados**. Isso permite que buttons, text buttons, checkboxes, etc, possam ser usados como **radio buttons**.
- **TextField**: é uma entrada de texto de uma linha.
 - Possui drawables para o **fundo**, **cursor de texto**, **seleção de texto**, uma fonte, fonte e cor da fonte para texto inserido, e fonte e cor da fonte para a mensagem que é mostrada quando está vazio.
 - Modo **password** pode ser ativado (mostrando asteriscos).



WIDGETS

- **TextArea**: similar ao **TextField** mas permite mais de uma linha para input de texto.
- **List**: desenha itens textuais e **destaca o item selecionado**.
 - **List** tem fonte, background para o item selecionado, e cor de fonte para os itens selecionados e não selecionados. A lista não tem scroll por si só mas pode ser colocada dentro de um **ScrollPane**.
- **SelectBox**: é uma lista drop-down.
 - Permite que um dos valores da lista **seja selecionado**. Quando inativo, o valor selecionado é desenhado. Quando ativa, mostra as opções em dropdown.
 - Possui drawables para o background, background para a lista, fonte e cor da fonte. Além disso possui atributo para espaçamento que deve ser respeitado entre os itens.



WIDGETS

- **ProgressBar**: desenha visualmente o progresso ou completude de algo (barra de progresso).
 - Possui um **range (mínimo e máximo)** e uma discretização dos valores. Pode ser **horizontal ou vertical**. A **direção do crescimento** também pode ser controlada. É possível **ativar animações de crescimento** da barra para que o crescimento tenha um efeito mais suave.
- **Slider**: slider é um campo horizontal que permite que o usuário **escolha um valor** (como uma barra de volume por exemplo).
 - Slider tem min,max, um step (para discretização dos valores), tem drawables para o background, o knob, e para a porção do slider antes e depois do knob.
- **Window**: é uma Table com uma **barra de título**.
 - Pode ser utilizada como uma caixa de diálogo.



WIDGETS

- **TouchPad**: é um joystick visual que move numa área circular.
 - Tem drawable para o background, e para o knob que o usuário movimenta.
- **Dialog**: é uma window com uma Table e um botão embaixo.
- **Observação**: é possível utilizar drag/drop entre os atores mas não vamos entrar nesse quesito aqui na disciplina.



SKINS

- A classe Skin **armazena recursos** para utilização nos widgets.
 - Contém uma **pré-definição** de texture regions, ninepatches, fonts, colors, etc.
- Métodos convenientes:

```
Skin skin = ...  
Color red = skin.getColor("red");  
BitmapFont font = skin.getFont("large");  
TextureRegion region = skin.getRegion("hero");  
NinePatch patch = skin.getPatch("header");  
Sprite sprite = skin.getSprite("footer");  
TiledDrawable tiled = skin.getTiledDrawable("pattern");  
Drawable drawable = skin.getDrawable("enemy");
```


CONVERSÕES

- Também é possível fazer:

```
Skin skin = new Skin();
skin.add("logo", new Texture("logo.png"));
...
Texture texture = skin.get("logo", Texture.class);
TextureRegion region = skin.getRegion("logo");
NinePatch patch = skin.getPatch("logo");
Sprite sprite = skin.getSprite("logo");
TiledDrawable tiled = skin.getTiledDrawable("logo");
Drawable drawable = skin.getDrawable("logo");
```

EXEMPLOS DE UTILIZAÇÃO

- Criando um botão a partir de uma skin:

```
TextButtonStyle buttonStyle = skin.get("bigButton", TextButtonStyle.class);  
TextButton button = new TextButton("Clique aqui!", buttonStyle);
```

- Alternativamente, também é possível fazer:

```
TextButton button = new TextButton("Clique aqui!", skin, "bigButton");
```

- Se o nome do estilo for omitido, o padrão "default" é utilizado, como no seguinte caso:

```
TextButton button = new TextButton("Clique aqui!", skin);
```



SKINS

- Vocês podem encontrar diversas skins nos sites (com exemplos):
 - <https://ray3k.wordpress.com/artwork/>
 - <https://github.com/czyzby/gdx-skins> (entrar nas pastas)
- Existe também um criador de skins, caso tenham interesse futuro:
 - https://www.youtube.com/watch?v=78amAV0_e24



EXERCÍCIO

- Agora, diferente do exercício anterior, utilize as **classes padrões da libgdx**, **Stage**, **Actor** e / ou **Group** para montar uma tela de menu de boas-vindas para o código do seu jogo que você vem montando ao longo da disciplina.